Building Decoder-Only Language Models for Hindi and Nepali

Language Models and Agents, Monsoon 2026 — Individual Project Report

Sohail Memon
Roll Number: 2025201075

Overview

This report covers the three phases of my individual project: building two from-scratch, decoder-only Transformer language models in Hindi and Nepali. I refer to the Hindi model as Model H and the Nepali model as Model L throughout, following the higher-resource/lower-resource framing the assignment asks for. Both models share an identical architecture and hyperparameters and are trained completely independently, so any gap I observe between them is a statement about the languages and data, not about one model being a different shape than the other.

Phase 1 covers how I collected and cleaned roughly 500 million tokens of text per language and built an independent tokenizer for each. Phase 2 covers the architecture decisions behind the model itself, how I pretrained it on Kaggle's T4 GPUs, and what I found when I evaluated the resulting checkpoints. Phase 3 covers a synthetic comparative-reasoning task I designed to finetune and probe both models further, and closes with a consolidated comparison answering how the two languages actually differ once the data is accounted for. This report keeps to what I consider the load-bearing decisions and results from each phase; for the full experimental detail behind any of it, I keep a separate, complete report per phase in the project repository (`phase1_data_report.md`, `phase2_model_report.md`, `phase3_reasoning_report.md`), which is the place to look for anything this report had to compress or leave out.

1 Phase 1: Data Collection, Preprocessing, and Tokenizers

1.1 Language choice and the non-overlap problem

I picked Hindi as my higher-resource language: it has the largest Wikipedia and news-crawl footprint of any Indic language and the biggest single split in IndicCorpV2. For the lower-resource language I picked Nepali from the assignment's allowed list, mainly because it shares the Devanagari script with Hindi (so I could reuse the same script-purity and safety-filtering tooling) and still has an active news ecosystem to scrape.

The assignment asks for at least 20% of each corpus to be manually collected, and for that manual portion to be provably free of overlap with whatever public data I also used. Rather than trying to prove non-overlap after the fact through deduplication, I built the guarantee in structurally: IndicCorpV2's underlying data is frozen around October–November 2022 (per its accompanying paper), so I scraped only articles published on or after 2023-01-01. A document that postdates the public corpus's collection window cannot be in it.

1.2 Collecting the data

For the public side, I streamed a prefix of IndicCorpV2's Hindi and Nepali files rather than downloading the full multi-gigabyte corpus, since the files are pre-shuffled (documented in AI4Bharat's own pipeline), so a prefix is already a valid random sample. For manual collection I wrote a sitemap-based scraper: discover a domain's article URLs from its sitemap, filter to the 2023 cutoff, fetch with `httpx`, and extract the article body with `trafilatura`. Six Hindi sources only exposed a rolling window of recent articles – nowhere near enough volume – until I found that abplive.com also has a dated daily archive back to October 2023, which closed the gap on its own. On the Nepali side, one source's sitemap `lastmod` field turned out to be identical across every article regardless of real publish date (caught when an article it claimed was from 2023 turned out to be a 2016 sports story); I fixed this, and a similar case elsewhere, by trusting the publish year baked

into the URL path instead.

Manual collection ended at 199,988 Hindi articles (~130.0M tokens) and 313,872 Nepali articles (~122.7M tokens), across 7 and 4 working sources respectively.

1.3 Cleaning, deduplication, and the token budget

Every document passes through Unicode NFC normalization first, then a pipeline of: line-level script-purity filtering (dropping individual lines under 75% Devanagari characters, rather than checking the whole document, so a single English heading doesn't sink an otherwise-clean article); length and symbol-ratio quality filters; a hand-curated Devanagari safety filter (I built this by transliterating a reviewed blocklist and then deliberately removing words with legitimate everyday meanings that happened to double as slang); exact deduplication by content hash, applied for the first time across manual and public sources combined; paragraph-level deduplication for shared boilerplate; and a corpus-wide MinHash near-duplicate pass. The MinHash stage is the most telling diagnostic in this phase: it removed 0.55% of Hindi's documents as near-duplicates versus 4.58% of Nepali's – about eight times higher, which lines up with Nepali's manual data coming from fewer, more concentrated news outlets that often cover the same stories.

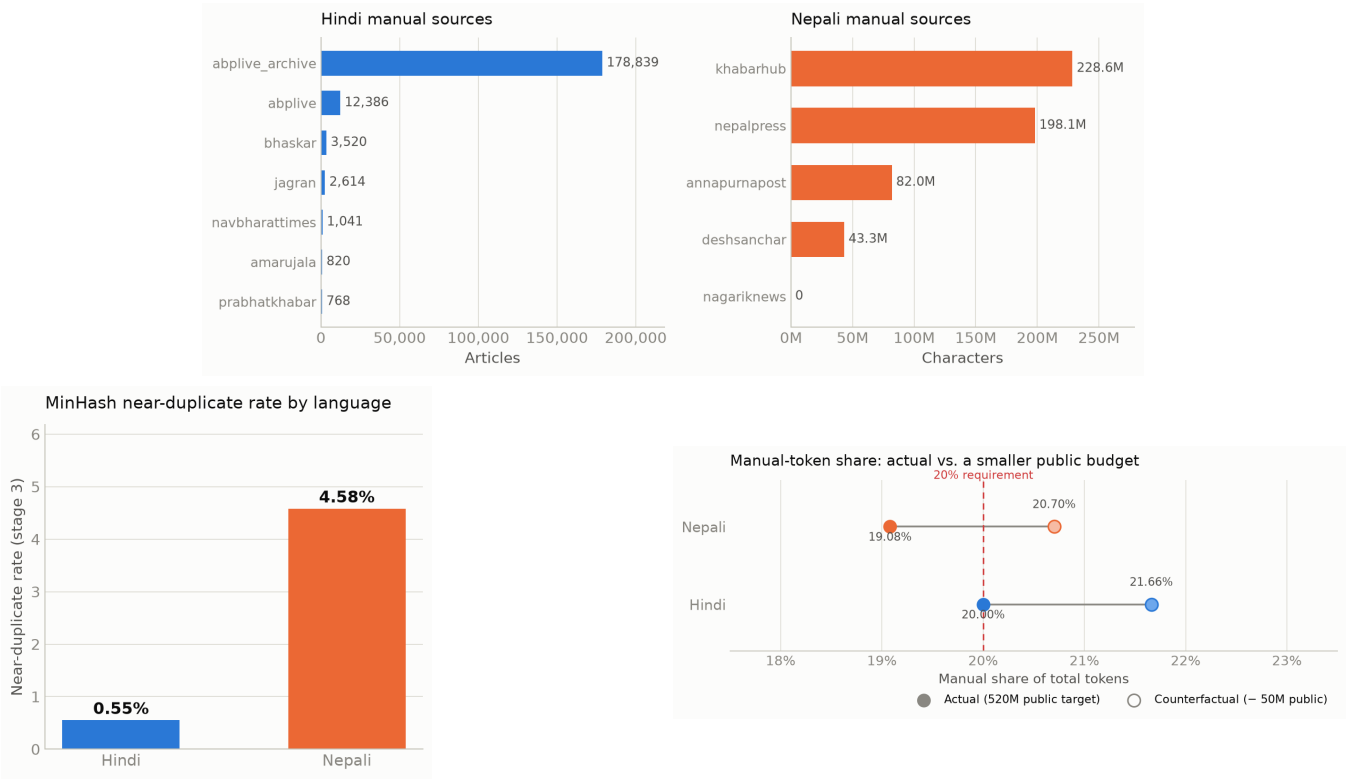


Figure 1: Top: per-source article/character counts for manual collection. Bottom left: final MinHash near-duplicate rate. Bottom right: manual-token share against the 20% requirement, actual vs. a smaller-public-buffer counterfactual.

I split the final corpus 95/2.5/2.5 into train/val/test, then capped it at roughly 510M tokens per language so both languages land close to the assignment's ~500M-token target. On raw collected tokens, manual data looks right at or just under the 20% requirement (20.00% Hindi, 19.08% Nepali) – but that undercounts a deliberately over-provisioned public buffer; net of a realistically sized buffer, both languages clear 20% comfortably.

1.4 Tokenizer: choosing 16,000 over the fertility-optimal vocabulary

I trained an independent SentencePiece Unigram tokenizer per language – no shared vocabulary, no pre-trained tokenizer. The assignment asks for the vocabulary size to be chosen using a fertility/UNK-rate

Language	Train docs	Total chars	Actual tokens (measured)
Hindi	4,581,781	2,040,000,247	~483.2M
Nepali	6,387,191	2,295,000,003	~477.2M

Table 1: Final delivered corpus, after the full preprocessing pipeline and 510M-token cap.

sweep on held-out text, so I trained seven candidate sizes from 4,000 to 48,000 per language and measured both metrics on a held-out sample. Fertility drops steeply through 16k–24k and then flattens for both languages, which on its own would favor something closer to 24k–32k.

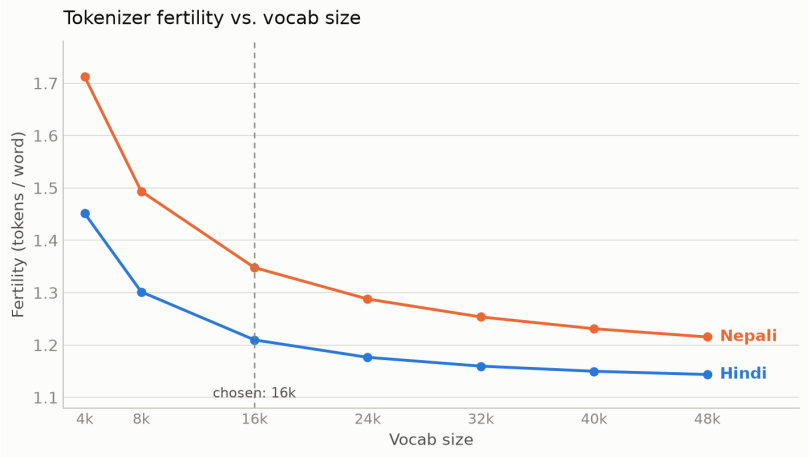


Figure 2: Fertility (tokens/word) vs. vocabulary size, 4k–48k, both languages. Both curves flatten past 16k.

I chose 16,000 instead, once I looked ahead to Phase 2: vocabulary size sets the size of the model's tied embedding table, and at the modest hidden width I planned to use, a 32k vocabulary would have put roughly half the entire parameter budget into that one table – well above what comparable small language models spend on embeddings. The fertility cost of dropping from 32k to 16k is real but small (about 0.05–0.09 extra tokens per word), so I took the trade. My final 16k tokenizers land at fertility 1.209 (Hindi) and 1.348 (Nepali) tokens/word, with UNK rates under 0.25% for both – good coverage of held-out text at this vocabulary size. Nepali's higher fertility here is consistent with its smaller, more repetitive collected corpus training a slightly less efficient subword vocabulary at the same budget.

2 Phase 2: Architecture, Pretraining, and Evaluation

2.1 Architecture: how the tokenizer decision fed back into the model

Both models share one architecture, trained independently. My first candidate – 384-dimensional hidden width, 7 layers – paired with Phase 1's original 32k-vocab tokenizer, and checking the parameter breakdown surfaced the same problem that drove the tokenizer decision above: at that width, the tied embedding table alone was 49.7% of the whole model, well past what reference small language models spend on embeddings (GPT-2-small sits at 31%). That finding is what actually forced the tokenizer down to 16k; Phase 1 and Phase 2 were not decided in isolation from each other.

With vocabulary fixed at 16,000, I swept hidden width against depth for a fixed ~25M-parameter budget. Wider models fit dramatically fewer layers, because a Transformer block's cost scales quadratically with hidden width while the embedding only scales linearly – at width 768, the budget only fits one or two layers. I resolved this in favor of depth, following Kaplan et al. (2020), who found loss depends only weakly on shape at a fixed parameter count, and Levine et al. (2020) and Tay et al. (2021), who both argue that models in this general size range tend to be too shallow rather than too deep. I settled on hidden width

384 with 11 layers and 6 attention heads (25,663,872 parameters total), the deepest option within 3% of my target budget, keeping the embedding table down to a healthier 23.9% of the model.

For positional information I used RoPE (Rotary Position Embeddings) rather than sinusoidal or learned absolute encodings. RoPE rotates the query and key vectors inside every attention layer based on token position, so every layer gets a direct signal about relative distance between tokens, rather than the input-only signal sinusoidal encoding provides that later layers have to infer distance from indirectly. It also carries no fixed-size position table, so it has no hard sequence-length ceiling the way a learned absolute embedding does, even though this project always trains at a fixed context length of 512 regardless. Like sinusoidal encoding, RoPE adds zero learned parameters, so this choice didn't touch the parameter budget above. I also tied the input and output embeddings (an untied head would cost another full embedding table for a transform I'd already learned once) and used pre-norm placement for more stable gradients through an 11-layer residual stack.

Hyperparameter	Value
Vocabulary	16,000 per language
Hidden width / layers / heads	384 / 11 / 6
Feed-forward width	1,536 (4× hidden width)
Context length	512
Positional encoding	RoPE
Dropout	0.1
Parameters	25,663,872

Table 2: Final architecture, shared by both models.

2.2 Pretraining

I tokenized each language's train/val/test text once into a flat, memory-mapped binary file of token ids, joined end to end by the tokenizer's end-of-sequence marker, which is what made working with a corpus this size practical without loading it all into RAM. Training samples a fresh, randomly-positioned window on every step rather than walking fixed non-overlapping chunks, the same convention nanoGPT uses; this makes "two epochs" a statement about expected coverage rather than a guarantee that every token is seen exactly twice, a simplification I kept given the scale of this project rather than building the indexing machinery that would guarantee exact coverage.

Training itself uses AdamW with the standard GPT-style split (weight decay on 2D+ parameters only, none on biases or norm parameters), a linear warmup into cosine decay, and gradient-norm clipping at 1.0. My first attempt at an effective batch size of 64 ran out of memory on a Kaggle T4 during the very first backward pass – the model's own parameters are tiny at this scale, but activation memory across 11 layers at that batch size and a 512-token context was not. I fixed this with gradient accumulation: a micro-batch of 16 accumulated over 4 steps, keeping the effective batch size and gradient statistics unchanged at roughly a quarter of the peak memory. I sized the number of training iterations from each language's actual measured token count for two epochs, which gave Hindi 28,354 iterations and Nepali 28,190 – close enough to identical that both land at a tokens-per-parameter ratio of about 18, just under the ratio Hoffmann et al. (2022) found compute-optimal.

I trained both languages in parallel on separate GPUs of a Kaggle T4×2 instance, logged to Weights & Biases, and pushed every checkpoint to a private Hugging Face Hub repository so a re-launched notebook could resume automatically – necessary because my ~17-hour estimated wall-clock time exceeds Kaggle's ~12-hour session cap. That estimate held: the first session ran 11h58m before being cut off at iteration ~19,600 (Hindi) / ~19,250 (Nepali), auto-resumed cleanly from the last saved checkpoint at iteration 19,000, and a second session finished the remaining iterations, for a total of ~17h37m (Hindi) and ~17h51m (Nepali).

Both curves show the shape I expected from a from-scratch run: a steep drop from the random-initialization loss down to about 5 within the first thousand iterations, then a long, still-descending tail through iteration

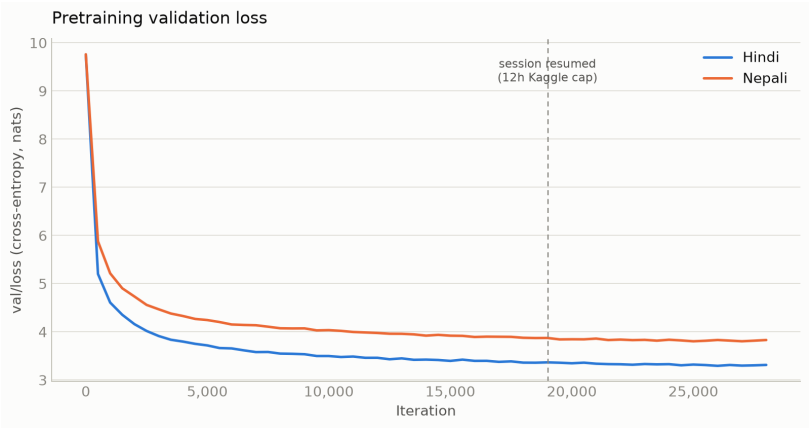


Figure 3: Pretraining validation loss for both languages across the full run.

28,000, with train and validation loss tracking closely the whole way – no sign of overfitting at this token budget. There's no visible discontinuity at the session-resume boundary, which told me the checkpoint and optimizer-state resume was working correctly. Nepali's loss sits above Hindi's for the entire run despite identical architecture and hyperparameters, which is consistent with Nepali being the lower-resource language rather than anything about the training setup.

2.3 Evaluation

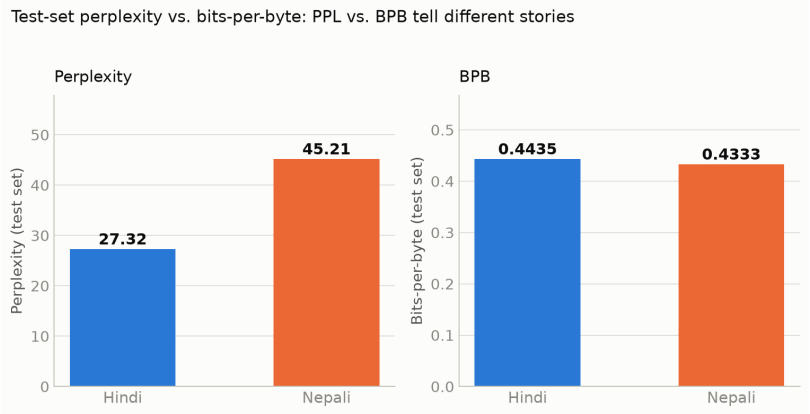


Figure 4: Test-set perplexity vs. bits-per-byte, Hindi and Nepali.

On the held-out test set, Hindi reaches perplexity 27.32 and Nepali 45.21 – at face value a large gap. But measured in bits-per-byte, which divides by the number of bytes in the underlying text rather than by the number of tokenizer tokens, Hindi and Nepali come out essentially tied (0.4435 vs. 0.4333, Nepali marginally *better*). The reason is tokenizer fertility: Nepali's tokenizer splits words into more, shorter pieces, and more tokens per word mechanically lowers per-token cross-entropy without the model having learned anything differently, since more of each word is already determined by the tokens already seen earlier in it. Bits-per-byte cancels that effect out, and most of the raw perplexity gap goes with it.

I generated continuations at four decoding settings (greedy, and temperature 0.5, 1.0, 1.5) from 50 held-out 32-token prefixes, and scored the 64-token continuations against reference text with BLEU-4 and chrF++, plus two reference-free diagnostics: 4-gram repetition rate and Distinct-1/2 (the fraction of generated 1-grams/2-grams that are unique).

The shape is exactly what I'd expect from a small language model, and it's the same shape for both languages: greedy decoding gets the closest match to the reference (highest BLEU-4) but is also by far the most repetitive – 33-43% of its own 4-grams repeat, the classic failure mode where a small model

Hindi

Setting	BLEU-4	chrF++	Rep.	Dist-2
Greedy	2.75	16.87	0.330	0.347
Temp 0.5	2.25	19.43	0.093	0.617
Temp 1.0	0.83	17.99	0.004	0.874
Temp 1.5	0.16	15.17	0.000	0.984

Nepali

Setting	BLEU-4	chrF++	Rep.	Dist-2
Greedy	0.63	13.67	0.431	0.185
Temp 0.5	0.61	16.29	0.057	0.651
Temp 1.0	0.29	16.36	0.000	0.924
Temp 1.5	0.09	15.58	0.000	0.993

Table 3: Generation quality across decoding settings. Rep. = 4-gram repetition rate; Dist-2 = Distinct-2.

gets stuck cycling a handful of high-probability tokens. Raising temperature monotonically fixes repetition and diversity (Distinct-2 climbs from around 0.2-0.35 at greedy to over 0.98 at temperature 1.5) at the direct cost of reference similarity, since the output drifts further from any one specific continuation the more randomness I inject. chrF++ is the interesting exception: it peaks at temperature 0.5 rather than at greedy for both languages – a small amount of sampling escapes greedy's repetition loops without yet drifting far enough to lose character-level similarity. I trust chrF++ more than BLEU-4 here because it credits shared character n-grams rather than whole-token matches, which matters for Hindi and Nepali: both carry case, postposition, and verb-agreement suffixes that change a word's surface token without changing its root.

Finally, I looked at attention patterns across layers using the raw post-softmax attention weights (recomputed manually, since the fused attention kernel used during training never materializes them), averaged over several example sentences per language.

	Hindi L0	Hindi L10	Nepali L0	Nepali L10
Mean entropy (nats)	1.236	0.993	1.743	1.073
Mean attn. distance	1.683	2.184	3.330	4.697

Table 4: Attention entropy and mean attention distance, layer 0 (early) vs. layer 10 (late).

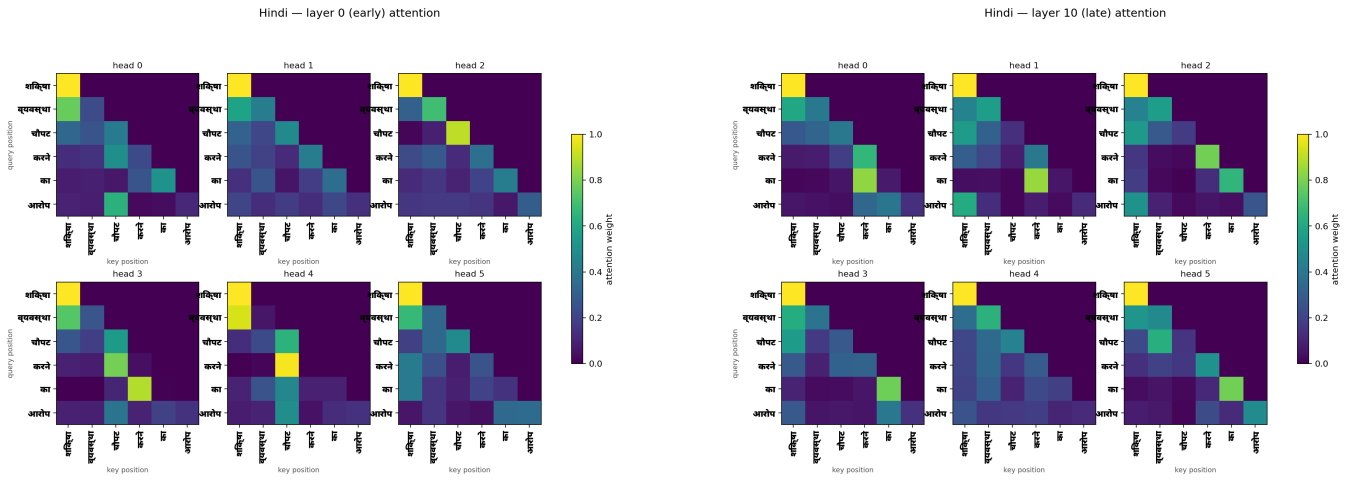


Figure 5: Hindi attention heatmaps, all heads: layer 0 (left, diffuse) vs. layer 10 (right, focused) – the entropy/distance numbers above, visualized.

Entropy falls and attention distance rises from the first to the last layer for both languages – early layers spread attention diffusely over nearby tokens, consistent with local, syntactic composition, while later layers commit more sharply to fewer positions and reach further back, consistent with integrating longer-range context using the structure the early layers already built. Individual heads specialize rather than layers doing so uniformly: within a single layer I found entropy varying by more than a factor of two across

heads (Hindi's layer 3 alone ranges from 0.53 to 1.26 nats across its six heads). The clearest example of a purely local head is Hindi's layer 3, head 3 – mean distance 1.11, essentially always attending to the token immediately before it, with the lowest entropy of any head in the model. The clearest content-based long-range head is Nepali's layer 8, head 2 – the lowest entropy of any head in either model (0.33 nats) combined with the longest mean distance (6.77), meaning it commits almost entirely to one specific position even when that position is far away. Both behaviors coexisting in the same model tells me specialization here is a head-level property, not something a whole layer does uniformly.

3 Phase 3: Reasoning Finetuning, Attention, and Final Comparison

3.1 A synthetic comparative-reasoning dataset

Rather than using an existing benchmark, I wrote a generator (`generate_reasoning_data.py`) that produces comparative-reasoning problems in three categories: `direct_numeric` (explicit numeric attributes for 2–3 entities), `chain_transitive` (two relational facts stated about 3 entities, with a question about the two *non-adjacent* entities that forces genuine transitive inference rather than a directly-stated answer), and `chain_multihop` (three chained facts across 4 entities). Each example asks either which entity is largest/smallest (`which`, answered with an entity name) or how two entities compare (`relation`, answered with one of three fixed words). I deliberately assign values to entities in mention order rather than sorted order, so a model can't shortcut the task by always picking the first-named entity – it actually has to read the numbers or chain the stated relations.

Leakage control runs on two independent axes: each language's pool of entity names is split disjointly 70/15/15 across train/val/test, so a test-time name never appears in training, and every (category, form) combination is built from four distinct phrasing templates split disjointly the same way, so test-time sentence structures are also never seen during training. I used Claude to draft the Hindi vocabulary and templates, and Gemini 3.5 Flash for the Nepali ones (chosen after checking a benchmark showing frontier Gemini models retain substantially more accuracy on Nepali specifically than smaller open models); I reviewed both by hand for grammatical consistency before fixing them into the generator. I settled on 16,000/2,400/2,400 examples per language – four times my initial pass, for reasons in the next section – and independently re-derived every example's gold answer from its own stored values to check the generator itself: zero mismatches across all 41,600 generated examples.

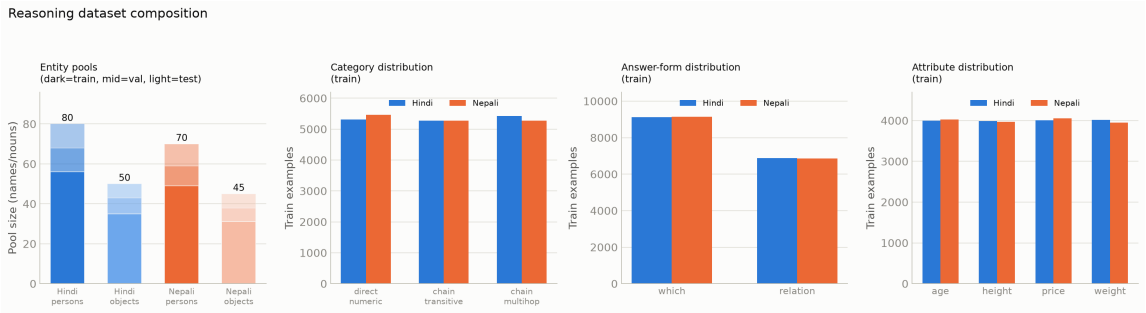


Figure 6: Reasoning dataset composition: entity-pool sizes and category/form/attribute balance.

A `chain_transitive/relation` example from each language, to make the design concrete:

> पंखा का वज़न नींबू से कम है। साथ ही, नींबू का वज़न साइकिल से ज़्यादा है। पंखा और साइकिल की तुलना करें तो पंखा का वज़न साइकिल से ज़्यादा है, कम है, या बराबर है? → **कम**
>
> लसुनको तौल सिरानीको भन्दा कम छ। सिरानीको तौल टेबलको भन्दा बढी छ। लसुन र टेबलबीचको तुलना गर्दा लसुनको तौल टेबलको भन्दा बढी, कम वा बराबर छ? → **बढी**

3.2 Finetuning: two rounds, both driven by what the numbers actually showed

Each model starts from its own Phase 2 best checkpoint, keeps its tokenizer fixed, and finetunes independently with a fresh AdamW optimizer (no carried-over momentum from pretraining) using prompt-loss-

masked supervised finetuning – only the answer tokens contribute to the loss, which needed no change to the model itself since causal attention already guarantees a real token can't attend to the padding after it.

Hyperparameter	Value
Epochs / batch size	8 / 32
Peak LR / min LR	3e-5 / 3e-6
Warmup iters	50 (of 4,000 total)
Weight decay / grad clip	0.1 / 1.0
Precision	bfloat16
Wall-clock (per language)	~2-3 min

Table 5: Finetuning hyperparameters, both languages.

My first pass used 4,000/600/600 examples and checked validation loss once per epoch. Validation loss bottomed out after the very first epoch and rose for the rest of an 8-epoch schedule, but a closer look at accuracy showed loss and task performance weren't tracking each other cleanly: the more "overfit" final checkpoint actually scored higher on exact-match accuracy than the lowest-validation-loss checkpoint, with a large swing between the two answer forms. I regenerated the dataset four times larger and added a flag to check validation loss every 50 iterations instead of only at epoch boundaries, since a bigger dataset means a single epoch now covers far more gradient steps and an epoch-level check was simply too coarse to catch the real optimum. Doing only the first change without the second actually made things worse – one answer form's accuracy collapsed, because "epoch 0" on the larger dataset now ran straight past the true optimum before the first check ever happened. With both changes together, the real best checkpoint turned out to sit at iteration ~100 of a 4,000-iteration schedule, and overall accuracy improved for both languages over my first attempt.

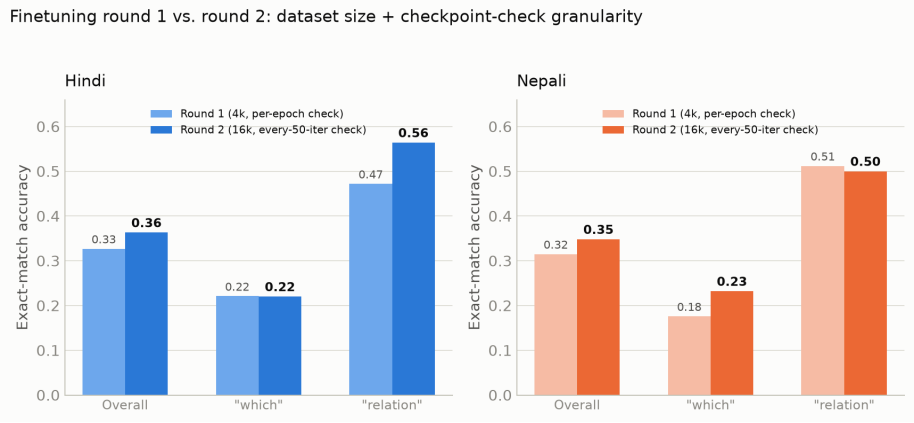


Figure 7: Exact-match accuracy: my first finetuning pass vs. the corrected one (larger dataset, finer-grained checkpoint checking).

3.3 Reasoning evaluation and what the errors look like

Zero-shot accuracy on this task is exactly 0% for both languages before finetuning, which is expected – the pretrained model was never trained to produce this fixed answer format at all. After finetuning, overall exact-match accuracy reaches 36.3% for Hindi and 34.8% for Nepali, close enough to each other that I don't read it as a real capability gap between the two languages so much as the same recipe producing a similar amount of task adaptation for both.

The split by answer form is the more informative result: `relation`, a fixed 3-way classification over a small and frequently-seen vocabulary, scores more than double `which`, which requires the model to correctly reproduce a specific entity name from its context – a genuinely harder generation constraint, especially

	Hindi	Nepali
Overall (finetuned)	36.3%	34.8%
`which` (entity name)	22.0%	23.2%
`relation` (3-word)	56.4%	50.0%

Table 6: Finetuned reasoning accuracy, overall and by answer form. Pretrained accuracy is 0.0% for both languages on every split.

since the held-out name split means the exact test-time names were never seen attached to this task during training.

Category	Hindi	Nepali
`direct_numeric`	32.2%	32.2%
`chain_transitive`	42.5%	40.7%
`chain_multihop`	34.1%	31.5%

Table 7: Finetuned accuracy by reasoning category.

`chain_transitive` scores highest for both languages rather than lowest, even though it most obviously demands multi-step inference – its relation-form question always asks about the same fixed pair of entities (the two ends of the chain), a narrower, more learnable pattern than `direct_numeric`'s open-ended comparison or `chain_multihop`'s randomly chosen pair among four entities. I'd rather note that than smooth it into a "harder categories score lower" story the data doesn't support. The finetuned model's `which` mistakes are almost always a real entity from the prompt rather than a fabricated one – it has learned to select from the given set, just not always the correct member, a qualitatively better failure mode than round 1, which more often invented a name absent from the prompt entirely. Two examples, Hindi correct and Nepali incorrect:

> ज्योति का कद कमला से कम है। साथ ही, कमला का कद नेहा से ज़्यादा है। बताइए, इनमें से किसका कद सबसे कम है? gold: ज्योति – predicted: ज्योति (correct)
>
> सुन्तलाको तौल अनारको भन्दा बढी छ। त्यस्तै, अनारको तौल कापीको भन्दा कम छ। त्यस्तै, कापीको तौल झोलाको भन्दा बढी छ। भन्नुहोस्, यी चारैमध्ये कसको तौल सबैभन्दा कम छ? gold: अनार – predicted: सुन्तला (a real entity, wrong one)

I also re-ran the Phase 2 attention analysis on both the pretrained and finetuned checkpoints against the same fixed set of six reasoning prompts, so any difference in the resulting heatmaps is attributable to finetuning rather than to different input text.

	Hindi pre.	Hindi fine.	Nepali pre.	Nepali fine.
Entropy, layer 0 (early)	2.285	2.259	2.429	2.409
Entropy, layer 10 (late)	1.665	1.555	1.655	1.829
Distance, layer 0 (early)	8.15	8.10	7.90	7.86
Distance, layer 10 (late)	9.75	10.60	8.42	7.27

Table 8: Attention entropy and mean distance on reasoning prompts, pretrained vs. finetuned.

The early-layer diffuse / late-layer focused pattern from Phase 2 persists essentially unchanged on these reasoning prompts, both before and after finetuning, and early-layer entropy/distance barely move after finetuning in either language (at most 0.03 nats, 0.05 tokens) – the early layers' role looks essentially fixed by pretraining alone. The late layer is where finetuning actually leaves a mark, and the two languages move in opposite directions: Hindi's late-layer entropy drops further (1.665 to 1.555, a sharper, more committed head) while its mean distance *increases* (9.75 to 10.60, reaching further back) – a plausible signature of the late layer becoming a more decisive long-range lookup, useful for pulling in a distant comparison target

across a multi-hop chain. Nepali moves the other way: entropy *rises* (1.655 to 1.829, more diffuse) while distance *drops* (8.42 to 7.27) – a late layer spreading attention more broadly over nearby tokens rather than reaching far, which doesn't fit the same reach-for-the-distant-entity story at all. Given the checkpoint sits so early in training (iteration ~100 of 4,000) and each run is independent, I read this as evidence that finetuning changes the late layer, direction depending on the specific run, not a single universal direction I'd claim from one run per language.

3.4 Final comparison: what actually explains the Hindi/Nepali gap

	Hindi (H)	Nepali (L)
Test perplexity (Phase 2)	27.32	45.21
Test bits-per-byte (Phase 2)	0.4435	0.4333
Reasoning accuracy, finetuned (Phase 3)	36.3%	34.8%

Table 9: Cross-phase summary. Pretrained reasoning accuracy is 0.0% for both.

Putting both phases together, the same pattern shows up twice: wherever a metric is sensitive to raw token counts – perplexity, and indirectly the tokenizer fertility that shapes it – Nepali reads noticeably worse. Wherever a metric normalizes past that – bits-per-byte in Phase 2, exact-match reasoning accuracy in Phase 3 – Hindi and Nepali land within a couple of points of each other. Hindi's manual data came from a larger, more varied set of sources than Nepali's, and Nepali's near-duplicate rate after deduplication was eight times Hindi's – a smaller, more concentrated publishing base to draw from. Architecture, training procedure, and the finetuning recipe were identical for both models throughout. So the gap I actually measured between a higher- and a lower-resource Indian language, at this model scale, looks substantially like a data-availability gap rather than a difference in what an identically-built model can learn from whatever data it gets.

References

[1] J. Su, Y. Lu, S. Pan, et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864, 2021.

[2] J. Kaplan, S. McCandlish, T. Henighan, et al. *Scaling Laws for Neural Language Models.* arXiv:2001.08361, 2020.

[3] Y. Levine, N. Wies, O. Sharir, H. Bata, and A. Shashua. *The Depth-to-Width Interplay in Self-Attention.* arXiv:2006.12467, 2020.

[4] Y. Tay, M. Dehghani, J. Rao, et al. *Scale Efficiently: Insights from Pretraining and Finetuning Transformers.* arXiv:2109.10686, 2021.

[5] J. Hoffmann, S. Borgeaud, A. Mensch, et al. *Training Compute-Optimal Large Language Models.* arXiv:2203.15556, 2022.

[6] S. Doddapaneni, R. Aralikatte, G. Ramesh, et al. *Towards Leaving No Indic Language Behind: Building Monolingual Corpora, Benchmark and Models for Indic Languages.* arXiv:2212.05409, 2022.